

Documentation Technique

Script PowerShell – Supervision de services Windows

PYRCZAK Noah | CIMPA – Sopra Steria | Stage Semaine 3

1. Contexte et objectif

Dans le cadre de la semaine 3 de stage chez CIMPA – Sopra Steria, j'ai développé un script PowerShell visant à assurer la disponibilité automatique des services critiques d'une machine Windows.

L'objectif est de :

- Détecter les services critiques qui ne sont pas en cours d'exécution.
- Tenter de les relancer automatiquement.
- Générer un rapport horodaté enregistré sur le Bureau.

2. Liste des services surveillés

Le script surveille les services suivants, identifiés comme critiques pour le bon fonctionnement du système :

Élément	Explication
W3SVC	Internet Information Services (IIS) – serveur web Windows
MSSQLSERVER	Microsoft SQL Server – moteur de base de données
Spooler	Gestionnaire d'impression Windows
BITS	Background Intelligent Transfer Service – transferts en arrière-plan
Dhcp	Client DHCP – attribution automatique d'adresses IP
TermService	Services Bureau à distance (RDP)
LanmanWorkstation	Service Workstation – accès aux ressources réseau partagées
Netlogon	Ouverture de session réseau – authentification sur le domaine
Dnscache	Client DNS – résolution des noms de domaine
RpcSs	Appel de procédure distante (RPC) – communication inter-processus
SysMain	Superfetch / SysMain – optimisation de la mémoire
Themes	Service de thèmes Windows – interface graphique
DoSvc	Service d'optimisation de livraison – mises à jour Windows

CertSvc	Services de certificat Active Directory – gestion des certificats SSL
----------------	---

3. Structure et explication du script

3.1 Initialisation

Le script commence par déclarer un tableau contenant les noms techniques des services à surveiller, puis initialise les variables de suivi.

Code – Initialisation
<pre>\$servicesAControle = @("W3SVC", "MSSQLSERVER", "Spooler", ...) \$rapport = @() \$dateExecutionScript = Get-Date \$statutEnCours = 'Running'</pre>

Élément	Explication
\$servicesAControle	Tableau (array) listant les noms exacts des services à surveiller
\$rapport	Tableau vide qui contiendra les résultats de chaque service traité
\$dateExecutionScript	Enregistre la date et l'heure de début d'exécution du script
\$statutEnCours	Variable contenant la valeur 'Running' pour éviter les erreurs de frappe (magic string)

3.2 Boucle de traitement – foreach

Le cœur du script est une boucle foreach qui parcourt chaque service de la liste. Pour chaque service, il vérifie son état et réagit en conséquence.

Code – Boucle foreach
<pre>foreach (\$svc in \$servicesAControle) { \$etat = Get-Service -Name \$svc -ErrorAction Stop if (\$etat.Status -ne \$statutEnCours) { Start-Service -Name \$svc -ErrorAction Stop \$ligne.Relance = "Tentative de relance effectuée" } else { \$ligne.Relance = "OK" } }</pre>

Élément	Explication
foreach	Itère sur chaque nom de service dans le tableau \$servicesAControle

<code>Get-Service -Name \$svc</code>	Récupère les informations du service (nom, statut, etc.)
<code>-ErrorAction Stop</code>	Force une erreur bloquante si le service est introuvable, capturée par le try/catch
<code>\$etat.Status</code>	Propriété contenant l'état actuel du service : 'Running', 'Stopped', etc.
<code>-ne \$statutEnCours</code>	Condition : si le statut est différent de 'Running', le service est arrêté
<code>Start-Service</code>	Commande PowerShell pour démarrer un service Windows

3.3 Gestion des erreurs – try/catch

Chaque appel à Get-Service et Start-Service est encapsulé dans un bloc try/catch pour gérer proprement les cas d'erreur sans interrompre le script.

Code – Gestion des erreurs	
<code>try {</code>	
<code> \$etat = Get-Service -Name \$svc -ErrorAction Stop</code>	
<code>}</code>	
<code>catch {</code>	
<code> Write-Warning "Le service '\$svc' n'a pas été trouvé ou est inaccessible."</code>	
<code> \$ligneErreur = [PSCustomObject]@{</code>	
<code> Service = \$svc</code>	
<code> Etat = "Introuvable/Inaccessible"</code>	
<code> Relance = "N/A"</code>	
<code> }</code>	
<code> \$rapport += \$ligneErreur</code>	
<code> continue</code>	
<code>}</code>	

Élément	Explication
<code>try { ... }</code>	Bloc d'essai : exécute le code et intercepte les erreurs
<code>catch { ... }</code>	Bloc de rattrapage : s'exécute si une erreur se produit dans le try
<code>Write-Warning</code>	Affiche un message d'avertissement en jaune dans la console
<code>[PSCustomObject]@{ }</code>	Crée un objet personnalisé avec des propriétés définies (Service, Etat, Relance)
<code>\$rapport += \$ligneErreur</code>	Ajoute la ligne d'erreur au tableau de rapport
<code>continue</code>	Passe immédiatement au service suivant dans la boucle foreach

3.4 Génération du rapport

Une fois tous les services traités, le script exporte les résultats dans un fichier texte horodaté sur le Bureau de l'utilisateur.

Code – Export du rapport

```
$cheminBureau = [Environment]::GetFolderPath("Desktop")
$nomFichier = "RapportServices_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').txt"
$cheminLog = Join-Path $cheminBureau $nomFichier
$rapport | Out-File -FilePath $cheminLog -Encoding UTF8 -Force
$rapport | Format-Table -AutoSize
```

Élément	Explication
<code>[Environment]::GetFolderPath("Desktop")</code>	Récupère dynamiquement le chemin du Bureau de l'utilisateur connecté
<code>Get-Date -Format 'yyyyMMdd_HH:mm:ss'</code>	Génère un horodatage unique pour éviter l'écrasement des anciens rapports
<code>Join-Path</code>	Combine le chemin du Bureau et le nom du fichier de façon sécurisée
<code>Out-File -Encoding UTF8</code>	Ecrit le contenu dans un fichier texte en encodage UTF-8 (gestion des accents)
<code>-Force</code>	Ecrase le fichier s'il existe déjà sans demander de confirmation
<code>Format-Table -AutoSize</code>	Affiche le rapport dans la console sous forme de tableau lisible

3.5 Calcul de la durée d'exécution

En fin de script, la durée totale d'exécution est calculée et affichée pour permettre un suivi des performances.

Code – Durée d'exécution

```
$dateFinExecution = Get-Date
$dureeExecution = $dateFinExecution - $dateExecutionScript
Write-Host "Temps d'exécution : $($dureeExecution.TotalSeconds) secondes."
```

Élément	Explication
<code>Get-Date</code>	Capture la date et heure actuelle à la fin du script
<code>\$dateFinExecution - \$dateExecutionScript</code>	Soustraction de deux objets DateTime, retourne un objet TimeSpan
<code>.TotalSeconds</code>	Propriété du TimeSpan retournant la durée totale en secondes

4. Compétences Bloc 1

Sous-compétence	Indicateur de performance
Vérifier les conditions de la continuité d'un service informatique	Les conditions de continuité sont vérifiées et les manquements sont signalés.
Traiter des demandes concernant les services réseau et système	Une solution à l'incident est trouvée et mise en œuvre.
Réaliser les tests d'intégration et d'acceptation d'un service	Des tests pertinents sont rédigés et effectués. Un rapport est produit.
Analyser les objectifs et les modalités d'organisation d'un projet	Les objectifs du projet sont explicités et les activités planifiées.

5. Difficultés rencontrées et solutions

Probleme rencontre	Solution apportee
Difficulté à identifier les noms exacts des services système	Utilisation de Get-Service pour lister et identifier les noms techniques corrects
Droits insuffisants pour redémarrer certains services	Exécution du script en tant qu'Administrateur (clic droit > Exécuter en tant qu'administrateur)
Fiabilité des relances non vérifiable en temps réel	Mise en place d'une boucle de log et d'un objet PSCustomObject pour tracer chaque tentative